📘 BOOK 14 — SPRING SECURITY

Authentication, Authorization & Production Security (Telugu + English)

Series: Java Interview + Development Mastery Series **Book Number:** 14 of 24 **Versions Covered:** Spring Security 6.x (lambda-based configuration DSL), JWT (`jjwt` /Nimbus libraries), Spring Boot 3.x **Prerequisites:** Book 10 (HTTP/authentication/authorization concepts, JWT structure), Book 11 (AOP — method security uses proxies), Book 12 (Spring Boot REST APIs — what we're securing) **Next Book:** `15_Java_Testing.md`

📖 How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

Telugu: Book 10 లో authentication/authorization **concepts** (JWT structure, RBAC, 401 vs 403) నేర్చుకున్నాము. ఈ పుస్తకం లో Spring Security ఆ concepts ని **production-grade framework** గా ఎలా implement చేస్తుందో నేర్చుకుంటాము — security filter chain (Book 10, Ch.4's filter concept meీద build అవుతుంది), password hashing, JWT-based stateless auth, method-level authorization, CORS, CSRF.

English: Book 10 taught authentication/authorization concepts (JWT structure, RBAC, 401 vs 403). This book teaches how Spring Security implements those concepts as a production-grade framework — the security filter chain (built directly on Book 10, Ch.4's filter concept), password hashing, JWT-based stateless authentication, method-level authorization, CORS, and CSRF.

🎯 Learning Objectives

1. Understand Spring Security's filter chain architecture.
2. Implement authentication with `UserDetailsService` and proper password hashing.
3. Implement authorization via roles/permissions and method security annotations.
4. Build full JWT-based stateless authentication.
5. Implement refresh token lifecycle management.
6. Understand OAuth2/social login concepts.
7. Understand and correctly configure CORS.
8. Understand CSRF and when it does/doesn't apply.
9. Apply production security practices.

📑 Table of Contents

Ch.	Title
3	Authorization: Roles, Permissions & Method Security
4	JWT-Based Stateless Authentication
5	Refresh Tokens & Token Lifecycle
6	OAuth2 & Social Login Concepts
7	CORS Deep Dive
8	CSRF Deep Dive
9	Production Security Practices
10	Mini Project — JWT-Secured REST API
—	Final Revision Notes, Cheat Sheet, Interview Bank, Revision Plans, Mastery Checklist

---

CHAPTER 1 — Spring Security Architecture: The Filter Chain

Telugu Explanation

Spring Security ఖచ్చితంగా Book 10, Ch.4's **Filter chain** concept meీదనే build అయ్యింది — ఒక **Security Filter Chain** (multiple `Filter` instances క్రమంగా) ప్రతి incoming request ని process చేస్తుంది, `DispatcherServlet` (Book 10, Ch.2) కి చేరకముందే. ప్రతి filter ఒక్క specific security concern handle చేస్తుంది: authentication, CSRF (Ch.8), CORS (Ch.7), authorization decision.

Professional English Explanation

Spring Security is built directly on Book 10, Ch.4's **Filter chain** concept — a **Security Filter Chain** (a sequence of multiple `Filter` instances) processes every incoming request before it even reaches `DispatcherServlet` (Book 10, Ch.2). Each filter handles one specific security concern: authentication, CSRF (Ch.8), CORS (Ch.7), authorization decision.

Diagram — The Security Filter Chain

```
Incoming Request
      |
      v
+-------------------------------------------------------------+
| SECURITY FILTER CHAIN (Book 10, Ch.4's filter concept, applied)|
|                                                               |
|  1. CorsFilter (Ch.7)                                         |
|  2. CsrfFilter (Ch.8)                                           |
|  3. UsernamePasswordAuthenticationFilter / JwtAuthFilter (Ch.4)     |
|       - extracts credentials, calls AuthenticationManager             |
|       - on success: populates the SecurityContext                       |
|  4. ExceptionTranslationFilter                                            |
|       - catches AuthenticationException -> 401, AccessDeniedException -> 403
|  5. AuthorizationFilter (formerly FilterSecurityInterceptor)                  |
|       - checks: is the authenticated user ALLOWED to access this URL?         |
|                                                                                  |
+-------------------------------------------------------------+
      |
      v (only if authenticated AND authorized)
DispatcherServlet -> @RestController (Book 12)
```

Java Code — Basic Security Configuration

```
import org.springframework.context.annotation.*;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
import static org.springframework.security.config.Customizer.withDefaults;

@Configuration                                                    // Book 11, Ch.2
class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())                                  // Ch.8 - disabled
here for a stateless JWT API
            .cors(withDefaults())                                            // Ch.7
            .authorizeHttpRequests(auth -> auth                               // the
AuthorizationFilter's rules, declared here
                .requestMatchers("/api/auth/**").permitAll()                     //
login/register - no auth needed
                .requestMatchers("/api/admin/**").hasRole("ADMIN")                // Ch.3 -
role-based
                .requestMatchers("/actuator/health").permitAll()                    // Book 12,
Ch.10
                .anyRequest().authenticated()                                        //
everything else needs authentication
            )
            .sessionManagement(session -> session

.sessionCreationPolicy(org.springframework.security.config.http.SessionCreationPolicy.STATELESS
))  // Ch.4
            .build();
        return http.build();
    }
}
```

Internal Working

- Spring Security's `SecurityFilterChain` is registered as **one single `Filter`** (a `DelegatingFilterProxy` / `FilterChainProxy`) within the standard servlet filter chain (Book 10, Ch.4) — internally, that one filter delegates to Spring Security's own internal ordered list of specialized filters; this is precisely why Spring Security integrates cleanly with the plain servlet model Book 10 taught, rather than being some entirely separate mechanism.
- The order of filters (and the order of `authorizeHttpRequests` rules) is **critical and evaluated top-to-bottom** — the **first matching rule wins**; `requestMatchers("/api/admin/**").hasRole("ADMIN")` must appear **before** a broader `anyRequest().authenticated()` rule, or the broader rule would match `/api/admin/...` requests first and never let the more specific admin-only restriction apply — this is a real, common, security-critical misconfiguration risk.
- `SessionCreationPolicy.STATELESS` tells Spring Security to **never create or use** an `HttpSession` (Book 10, Ch.3) for authentication state — every single request must independently prove its identity (via a JWT, Ch.4), directly implementing Book 10, Ch.7's observation that stateless token-

based auth avoids the cross-server session-sharing problem, at the framework-configuration level.

Real-World Example

Telugu: `authorizeHttpRequests` rules order తప్పుగా పెడితే (broad rule ముందు, specific rule తర్వాత), unintended గా admin-only endpoints అన్ని authenticated users కి open అయిపోవచ్చు — ఇది real, documented, security-critical misconfiguration category, code review లో ప్రత్యేకంగా చూడాల్సినది.
English: Getting the `authorizeHttpRequests` rule order wrong (a broad rule before a specific one) can unintentionally leave admin-only endpoints open to any authenticated user — a real, documented, security-critical misconfiguration category that code reviews specifically need to check for.

Interview Answer

"Spring Security integrates as a single servlet Filter (Book 10, Ch.4) containing an internal, ordered chain of specialized filters — handling CORS, CSRF, authentication (extracting and validating credentials), and authorization (checking if the authenticated user can access this specific URL) — before the request ever reaches DispatcherServlet. Rule order in `authorizeHttpRequests` is critical and evaluated top-to-bottom, first-match-wins, so specific rules must precede broader ones. `SessionCreationPolicy.STATELESS` disables session-based auth entirely, requiring every request to independently prove identity, typically via JWT (Ch.4)."

Cross Questions

- Q: How does Spring Security integrate with the plain servlet filter model from Book 10? → A: It registers as one servlet filter that internally delegates to its own ordered chain of specialized security filters — it's built on, not separate from, the standard filter mechanism.
- Q: Why does rule order matter in `authorizeHttpRequests`? → A: Rules are evaluated top-to-bottom, first-match-wins — a broad rule placed before a more specific one would incorrectly match first, potentially exposing something meant to be more restricted.
- Q: What does `SessionCreationPolicy.STATELESS` actually change? → A: Spring Security never creates or relies on an `HttpSession` for authentication state — every request must independently authenticate, typically via a token (JWT, Ch.4), rather than a session cookie (Book 10, Ch.3).

Tricky Questions

- Q: If `csrf().disable()` is used for a stateless JWT API, is that a security regression? → A: No — CSRF protection (Ch.8) specifically defends against attacks exploiting automatic cookie-based credential submission; a stateless API relying on an explicitly-attached JWT header (not automatically sent by the browser like a cookie) isn't vulnerable to CSRF in the same way, so disabling it there is appropriate, not a regression — this distinction is covered fully in Ch.8.
- Q: Does `permitAll()` on `/api/auth/**` mean those endpoints have zero security consideration at all? → A: No — it means no *authentication* is required to reach them, but they still need their

own careful security handling (rate limiting on login attempts, input validation, Book 12 Ch.5) — "no auth required" and "no security needed" are different things entirely.

Coding Exercise

L1: Set up a basic `SecurityConfig` with public, authenticated, and role-restricted endpoint rules. **L2:** Deliberately misorder two rules (broad before specific) and observe the unintended access. **L3:** Configure `SessionCreationPolicy.STATELESS` and verify no session cookie is set on responses. **L4 (Interview):** Explain how Spring Security's filter chain relates to Book 10's plain servlet filters. **L5 (Senior):** Review a `SecurityConfig` with misordered rules exposing an admin endpoint — identify and fix the issue. **L6 (Mastery):** Explain, from memory, why `SessionCreationPolicy.STATELESS` is the correct choice for a JWT-based API, connecting to Book 10, Ch.7.

CHAPTER 2 — Authentication: UserDetailsService & Password Hashing

Telugu Explanation

`UserDetailsService` Spring Security కి "ఈ username కి ఏ user details, ఏ hashed password, ఏ roles ఉన్నాయి" అని చెప్పే interface — మీ own database/repository కి connect చేయాల్సిన ఒక్క method. Passwords **ఎప్పుడూ plaintext గా store చేయకూడదు** — `PasswordEncoder` (బై default `BCryptPasswordEncoder`) వాడి **salted hash** store చేస్తారు.

Professional English Explanation

`UserDetailsService` is the interface telling Spring Security "for this username, here are the user details, hashed password, and roles" — one method connecting to your own database/repository. Passwords should **never be stored in plaintext** — `PasswordEncoder` (by default `BCryptPasswordEncoder`) stores a **salted hash** instead.

Java Code

```
import org.springframework.security.core.userdetails.*;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.context.annotation.*;
import org.springframework.stereotype.Service;
import java.util.*;

@Service
class CustomUserDetailsService implements UserDetailsService {
    private final UserRepository userRepository;                            // Book 13's
JpaRepository
    CustomUserDetailsService(UserRepository userRepository) { this.userRepository =
userRepository; }

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        AppUser user = userRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException("User not found: " +
username));

        return org.springframework.security.core.userdetails.User.builder()
                .username(user.getUsername())
                .password(user.getHashedPassword())                                 // ALREADY
HASHED - never plaintext, ever
                .authorities(user.getRoles().stream()
                        .map(role -> new SimpleGrantedAuthority("ROLE_" + role))     // Spring
convention: "ROLE_" prefix
                        .toList())
                .build();
    }
}

@Configuration
class PasswordConfig {
    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();                                      // salted,
adaptive-cost hashing
    }
}

@Service
class RegistrationService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;                                  //
injected, per Book 11's DI

    RegistrationService(UserRepository userRepository, PasswordEncoder passwordEncoder) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }

    void register(String username, String rawPassword) {
        String hashed = passwordEncoder.encode(rawPassword);                            //
hash BEFORE storing
        userRepository.save(new AppUser(username, hashed, Set.of("USER")));
        // 'rawPassword' is now DISCARDED - never logged, never stored, never sent anywhere
```

```
    }

    boolean verifyLogin(String username, String rawPassword) {
        AppUser user = userRepository.findByUsername(username).orElseThrow();
        return passwordEncoder.matches(rawPassword, user.getHashedPassword());             //
hash the ATTEMPT, compare hashes
    }
}
```

Internal Working

- **BCrypt is deliberately, adjustably slow** (a "work factor" parameter controls how many hashing rounds it performs) — this is a genuine security feature, not a performance flaw: it makes brute-force password-guessing attacks (trying millions of candidate passwords against a stolen hash) computationally expensive, unlike a fast general-purpose hash (like plain SHA-256) which would let an attacker try billions of guesses per second on modern hardware.
- BCrypt **automatically generates and embeds a random salt** into its output hash string — this is precisely why two users with the identical password produce **different** stored hash values, defeating precomputed "rainbow table" attacks that work against unsalted hashes; `passwordEncoder.matches(raw, storedHash)` extracts that embedded salt from the stored hash itself to correctly re-hash and compare the login attempt.
- `verifyLogin()` never decrypts anything (BCrypt is a one-way hash, not reversible encryption) — it re-hashes the login attempt's raw password using the same salt/algorithm parameters embedded in the stored hash, then compares the two resulting hash strings; this is exactly why even Spring Security/the application itself can never recover a user's actual original password from the database, only verify a given guess against it.

Real-World Example

Telugu: Real production data breaches లో, plaintext-stored passwords ఉన్న systems catastrophic గా affect అవుతాయి — ఒక్క DB dump తో అన్ని user passwords బయటపడతాయి. BCrypt hashed passwords ఉంటే, attacker కి brute-force cost చాలా ఎక్కువ, salt వల్ల rainbow tables కూడా పనిచేయవు. English: In real production data breaches, systems storing plaintext passwords are catastrophically affected — a single database dump exposes every user's actual password immediately. With properly BCrypt-hashed passwords, an attacker faces a genuinely expensive brute-force cost, and per-user salting defeats rainbow-table attacks entirely — this is why proper password hashing is treated as a non-negotiable baseline, not an optional hardening step.

Interview Answer

"`UserDetailsService` connects Spring Security to your own user data source via one method, `loadUserByUsername`. Passwords are never stored in plaintext — `PasswordEncoder` (BCrypt by default) produces a salted, deliberately slow (adjustable work-factor) hash, making brute-force attacks computationally expensive and defeating rainbow-table attacks via per-user salting. Login verification re-hashes the attempted password using the same embedded salt and compares hash values — the original password is never recoverable, even by the application itself, since BCrypt is one-way."

Cross Questions

- Q: Why is BCrypt deliberately slow, and why is that a feature, not a bug? → A: Deliberate slowness makes brute-force password-guessing attacks against a stolen hash computationally expensive — a fast hash would let an attacker try vastly more candidate passwords per second.
- Q: Why do two users with the identical password get different stored hash values? → A: BCrypt automatically generates and embeds a random salt per hash — this defeats precomputed rainbow-table attacks that work against unsalted hashing schemes.
- Q: Can the application ever recover a user's original plaintext password from its stored hash? → A: No — BCrypt is a one-way hash function, not reversible encryption; verification works by re-hashing a login attempt and comparing hash values, never by decrypting anything.

Tricky Questions

- Q: If a raw password is accidentally logged somewhere in the application (e.g., in a debug log statement, Book 12, Ch.9) before hashing, does using BCrypt anywhere else in the system still protect it? → A: No — that specific leak point completely bypasses BCrypt's protection entirely; proper hashing everywhere doesn't help if the raw password was captured in plaintext at any point in transit or logging, which is exactly why Book 12, Ch.9's "never log sensitive data" guidance matters as much as the hashing itself.
- Q: Is it acceptable to increase BCrypt's work factor arbitrarily high for maximum security? → A: Not without consideration — an excessively high work factor makes legitimate login verification slow too (a real user-experience and server-load cost), so the work factor is a deliberate, tunable trade-off between security margin and acceptable latency, not "higher is always strictly better with no cost."

Coding Exercise

L1: Implement a `UserDetailsService` backed by a simple in-memory or JPA-backed user store. **L2:** Register a user with a hashed password using `BCryptPasswordEncoder`, then verify login with both correct and incorrect passwords. **L3:** Verify (by inspecting the stored hash) that hashing the same password twice produces two different hash strings. **L4 (Interview):** Explain why BCrypt's deliberate slowness and built-in salting are both security features. **L5 (Senior):** Review a legacy system storing passwords as unsalted SHA-256 hashes — explain the risk and design a migration plan to BCrypt without forcing all users to reset passwords immediately. **L6 (Mastery):** Explain, from memory, exactly what `passwordEncoder.matches()` does internally, and why it never involves decryption.

CHAPTER 3 — Authorization: Roles, Permissions & Method Security

Telugu Explanation

Book 10, Ch.8 లో RBAC concept నేర్చుకున్నాము. Spring Security దీన్ని **URL-level** (`hasRole()` / `hasAuthority()` in `authorizeHttpRequests`, Ch.1) మరియు **method-level** (`@PreAuthorize` / `@Secured` — Book 11, Ch.8's AOP proxies వాడి) రెండు స్థాయిల్లో అమలు చేస్తుంది. Method security **defense in depth** కి ఉదాహరణ — controller layer లో check తప్పిపోయినా, service layer లో కూడా enforce అవుతుంది.

Professional English Explanation

Book 10, Ch.8 taught the RBAC concept. Spring Security implements it at **two levels**: **URL-level** (`hasRole()` / `hasAuthority()` in `authorizeHttpRequests`, Ch.1) and **method-level** (`@PreAuthorize` / `@Secured` — using Book 11, Ch.8's AOP proxies). Method security is a real example of **defense in depth** — even if a controller-layer check is missed, service-layer enforcement still applies.

Java Code

```
import org.springframework.security.access.prepost.*;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.context.annotation.Configuration;
import org.springframework.stereotype.Service;

@Configuration
@EnableMethodSecurity                                    // activates
@PreAuthorize/@PostAuthorize processing (AOP, Book 11 Ch.8)
class MethodSecurityConfig {}

@Service
class DocumentService {

    @PreAuthorize("hasRole('ADMIN')")                              // checked BEFORE the
method runs
    void deleteAnyDocument(Long documentId) {
        System.out.println("Admin deleted document " + documentId);
    }

    @PreAuthorize("hasAuthority('DOCUMENT_WRITE')")                  // permission-based,
finer-grained than role-based
    Document createDocument(String title) {
        return new Document(title);
    }

    // ABAC-style check (Book 10, Ch.8) - resource OWNERSHIP, not just role
    @PreAuthorize("#document.ownerUsername == authentication.name or hasRole('ADMIN')")
    void editDocument(Document document) {
        System.out.println("Edited: " + document.getTitle());
    }

    @PostAuthorize("returnObject.ownerUsername == authentication.name or hasRole('ADMIN')")
// checked AFTER, on the RETURN value
    Document getDocument(Long id) {
        return findDocumentSomehow(id);                                 // must fetch FIRST
to know the owner, hence PostAuthorize
    }

    private Document findDocumentSomehow(Long id) { return new Document("Sample"); }
}

record Document(String title) {
    String getTitle() { return title; }
    String ownerUsername = "ravi";                                        // simplified
for demo
}
```

Internal Working

- `@PreAuthorize` / `@PostAuthorize` are implemented via **exactly** Book 11, Ch.8's AOP proxy mechanism — the annotated method is wrapped by a proxy that evaluates the SpEL (Spring Expression Language) authorization expression **before** (`@PreAuthorize`) or **after** (`@PostAuthorize`) the real method executes, throwing `AccessDeniedException` (which Ch.1's `ExceptionTranslationFilter` converts to a `403 Forbidden` , Book 10, Ch.1) if the expression

evaluates to `false` — this means Book 11, Ch.8's **self-invocation limitation applies directly here too**: calling a `@PreAuthorize`-annotated method via `this.method()` bypasses the security check entirely, a genuinely serious, frequently-tested real vulnerability class if not understood.

- **`@PostAuthorize`** exists specifically for cases (like `getDocument()` above) where the authorization decision **depends on data only available after fetching the resource** (you need to know the document's owner before you can check "is this the owner or an admin") — the trade-off is that the method body **does** execute before the check, meaning `@PostAuthorize` alone shouldn't be relied on for operations with side effects that must never occur for an unauthorized caller (a read is safe; a write generally needs `@PreAuthorize` with a pre-fetchable check, or a two-step fetch-then-authorize-then-act pattern).
- **Defense in depth**: applying authorization checks at both the URL/controller level (Ch.1) AND the service/method level means a mistake or omission at one layer (e.g., a controller endpoint accidentally missing its own explicit check) doesn't automatically create a full security hole, since the service layer's own `@PreAuthorize` still enforces the rule independently — this directly connects to Book 10, Ch.8's tricky question about needing authorization checks at multiple layers.

Real-World Example

Telugu: Multi-tenant SaaS applications లో, `@PreAuthorize("#document.ownerUsername == authentication.name")` వంటి ownership-based checks — ఒక user తన own documents మాత్రమే edit చేయగలగాలి, ఇతరుల documents కాదు, role ఒక్కటే సరిపోదు (Book 10, Ch.8's RBAC-insufficient scenario కి direct real implementation). English: Multi-tenant SaaS applications rely on exactly this ownership-based `@PreAuthorize` pattern — a user must only be able to edit their own documents, not others', where role alone is insufficient — a direct, real implementation of Book 10, Ch.8's "RBAC alone is insufficient" scenario.

Interview Answer

"Spring Security enforces authorization at both URL level (`authorizeHttpRequests`, Ch.1) and method level (`@PreAuthorize` / `@PostAuthorize`, via Book 11, Ch.8's AOP proxy mechanism), providing real defense in depth. `@PreAuthorize` checks a SpEL expression before the method runs, throwing `AccessDeniedException` (→ 403) on failure; `@PostAuthorize` checks after, needed when the decision depends on the method's own return value, like resource ownership. Critically, the same AOP self-invocation limitation applies — calling a secured method via `this.method()` bypasses the check entirely, a genuinely serious vulnerability if not understood."

Cross Questions

- Q: Why does @PreAuthorize's self-invocation limitation matter for security specifically, not just correctness? → A: A developer might assume calling a secured method internally still enforces the check, when it silently doesn't — this can create an actual, exploitable authorization bypass if that internal call path is ever reachable with attacker-controlled input.
- Q: When would you need @PostAuthorize instead of @PreAuthorize? → A: When the authorization decision depends on data only known after fetching/computing the result (like a

resource's owner field) — you can't check "is this the owner" before you've loaded the resource to see who the owner is.

- Q: What's the practical benefit of enforcing authorization at both the URL and method level? → A: Defense in depth — a missed or buggy check at one layer doesn't automatically create a full security hole, since the other layer's independent check still applies.

Tricky Questions

- Q: Is @PostAuthorize safe to use for an operation that both reads AND deletes a resource in one method? → A: Generally not recommended — since the method body (including the delete) executes before the authorization check, an unauthorized caller's delete would have already happened by the time @PostAuthorize denies access; write/destructive operations should use @PreAuthorize with a pre-fetchable check, or a fetch-then-check-then-act pattern instead.
- Q: Does `hasRole('ADMIN')` in a SpEL expression automatically add the `"ROLE_"` prefix, or must it match exactly what's stored? → A: `hasRole()` automatically expects/adds the `"ROLE_"` prefix convention (matching `hasRole('ADMIN')` against a granted authority of `"ROLE_ADMIN"`); `hasAuthority()` does NOT add any prefix, expecting an exact match — a subtle, genuinely common source of "why isn't my role check working" confusion.

Coding Exercise

L1: Add `@PreAuthorize("hasRole('ADMIN')")` to a service method and verify both authorized and unauthorized access attempts. **L2:** Implement an ownership-based `@PreAuthorize` check using a method parameter, mirroring the `#document.ownerUsername` pattern. **L3:** Reproduce the self-invocation bypass: call a `@PreAuthorize`-secured method via `this.method()` and confirm the check is skipped. **L4 (Interview):** Explain the difference between @PreAuthorize and @PostAuthorize, with a concrete example needing each. **L5 (Senior):** Review a multi-tenant application's authorization design relying only on role checks — identify where ownership-based (ABAC-style) checks are also needed, per Book 10, Ch.8. **L6 (Mastery):** Explain, from memory, why the AOP self-invocation limitation (Book 11, Ch.8) is a genuine security vulnerability class here, not just a correctness footnote.

CHAPTER 4 — JWT-Based Stateless Authentication

Telugu Explanation

Book 10, Ch.7 లో JWT structure (header.payload.signature) నేర్చుకున్నాము. ఇప్పుడు దీన్ని Spring Security లో పూర్తిగా implement చేద్దాం: login endpoint credentials verify చేసి JWT issue చేస్తుంది; ఒక custom `Filter` (Book 10, Ch.4) ప్రతి తర్వాతి request లో JWT ని extract చేసి, validate చేసి, `SecurityContext` populate చేస్తుంది — session/cookie లేకుండా.

Professional English Explanation

Book 10, Ch.7 taught JWT structure (header.payload.signature). Now, the full Spring Security implementation: a login endpoint verifies credentials and issues a JWT; a custom `Filter` (Book 10, Ch.4) extracts and validates the JWT on every subsequent request, populating the `SecurityContext` — with no session/cookie involved.

Java Code

```
import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.*;
import org.springframework.stereotype.Component;
import java.security.Key;
import java.util.*;

@Component
class JwtService {
    private final Key signingKey = Keys.secretKeyFor(io.jsonwebtoken.SignatureAlgorithm.HS256);
// in real code: from config/secrets
    private final long expirationMs = 15 * 60 * 1000;
// 15 minutes - SHORT-lived (Ch.5)

    String generateToken(String username, List<String> roles) {
        return Jwts.builder()
                .setSubject(username)                                          // the "sub"
claim
                .claim("roles", roles)                                            // custom
claim - NEVER put secrets here (Book 10, Ch.7)
                .setIssuedAt(new Date())
                .setExpiration(new Date(System.currentTimeMillis() + expirationMs))
                .signWith(signingKey)                                                // the
cryptographic signature (Book 10, Ch.7)
                .compact();
    }

    String extractUsername(String token) { return parseClaims(token).getSubject(); }

    boolean isTokenValid(String token) {
        try {
            parseClaims(token);                                                        //
throws if signature invalid OR expired
            return true;
        } catch (JwtException | IllegalArgumentException e) {
            return false;
// tampered, expired, or malformed
        }
    }

    private Claims parseClaims(String token) {
        return Jwts.parserBuilder().setSigningKey(signingKey).build()
                .parseClaimsJws(token).getBody();
// verifies signature AS PART of parsing
    }
}

@Component
class JwtAuthenticationFilter extends jakarta.servlet.http.HttpFilter {
// Book 10, Ch.4's Filter concept
    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    JwtAuthenticationFilter(JwtService jwtService, UserDetailsService userDetailsService) {
```

```
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilter(HttpServletRequest req, HttpServletResponse resp, FilterChain
chain)
            throws java.io.IOException, ServletException {
        String authHeader = req.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
// Book 10, Ch.7's Bearer pattern
            String token = authHeader.substring(7);

            if (jwtService.isTokenValid(token)) {
                String username = jwtService.extractUsername(token);
                UserDetails userDetails = userDetailsService.loadUserByUsername(username);
// Ch.2

                var authToken = new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());
// no password needed here - already verified
                SecurityContextHolder.getContext().setAuthentication(authToken);
// NOW this request is "authenticated"
            }
        }
        chain.doFilter(req, resp);
// ALWAYS continue the chain (Book 10, Ch.4)
    }
}

@org.springframework.web.bind.annotation.RestController
@org.springframework.web.bind.annotation.RequestMapping("/api/auth")
class AuthController {
    private final JwtService jwtService;
    // ... AuthenticationManager injected for real credential verification, omitted for brevity

    AuthController(JwtService jwtService) { this.jwtService = jwtService; }

    @org.springframework.web.bind.annotation.PostMapping("/login")
    Map<String, String> login(@org.springframework.web.bind.annotation.RequestBody LoginRequest
request) {
        // In real code: authenticationManager.authenticate(...) FIRST, verifying
username+password (Ch.2)
        String token = jwtService.generateToken(request.username(), List.of("USER"));
        return Map.of("accessToken", token, "tokenType", "Bearer");
    }
}
record LoginRequest(String username, String password) {}
```

Internal Working

- `Jwts.parserBuilder().setSigningKey(signingKey).build().parseClaimsJws(token)` is the single call that does **all** the verification work Book 10, Ch.7 described conceptually — it recomputes the signature over the token's header+payload using the known signing key and compares it against the token's actual signature, throwing an exception on **any** mismatch (tampering) or if the `exp` claim indicates expiration — this is the concrete implementation of "tamper-evident, not secret" from Book 10, Ch.7.

- `JwtAuthenticationFilter` **always** calls `chain.doFilter(req, resp)` at the end, regardless of whether authentication succeeded — this is deliberate: an invalid/missing token doesn't reject the request at *this* filter; it simply leaves the `SecurityContext` unauthenticated, and the **later** `AuthorizationFilter` (Ch.1) is what actually decides, based on the endpoint's configured rules, whether an unauthenticated request is allowed through (for `permitAll()` endpoints) or rejected (401, for endpoints requiring authentication) — a clean separation of "who are you" (this filter) from "are you allowed here" (the authorization filter).
- Populating `SecurityContextHolder` (a `ThreadLocal`-based holder, Book 08's `ThreadLocal` concept, scoped per-request-thread) is what makes the authenticated user's identity/roles available throughout the rest of that request's processing — including to `@PreAuthorize` expressions (Ch.3) referencing `authentication.name`, without needing to manually pass the user object through every method call.

Real-World Example

Telugu: Real production APIs `Authorization: Bearer <token>` header పంపి, ఈ chapter యొక్క `JwtAuthenticationFilter` ఖచ్చితంగా ఇలాంటి filter (production code లో more robust error handling తో) ద్వారా authenticate అవుతాయి — session/cookie ఏమీ లేకుండా, ప్రతి request స్వతంత్రంగా authenticate అవుతుంది. English: Real production APIs sending `Authorization: Bearer <token>` are authenticated by exactly this kind of filter (with more robust production error handling) — no session or cookie involved, every request independently authenticating itself, precisely the stateless model Book 10, Ch.7 introduced.

Interview Answer

"JWT-based authentication uses a login endpoint issuing a signed token, and a custom Filter (Book 10, Ch.4) validating that token on every subsequent request by extracting the `Authorization: Bearer` header, verifying its signature and expiration, and populating Spring Security's `SecurityContext` (a ThreadLocal holder) with the authenticated user's identity and roles. The filter always continues the chain regardless of validation outcome — an invalid token simply leaves the request unauthenticated, letting the later authorization filter decide whether that's acceptable for the specific endpoint."

Cross Questions

- Q: What does `parseClaimsJws()` actually verify, and what happens on failure? → A: It recomputes and compares the token's cryptographic signature, and checks the expiration claim — throwing an exception on any tampering or expiration, which `isTokenValid()` catches and translates to `false`.
- Q: Why does JwtAuthenticationFilter always call chain.doFilter(), even for an invalid token? → A: To cleanly separate "who is this" (this filter) from "are they allowed here" (the later authorization filter, Ch.1) — an invalid token just leaves the request unauthenticated, and the authorization rules for that specific endpoint decide the actual outcome.
- Q: What is SecurityContextHolder, and why is it ThreadLocal-based? → A: It holds the current request's authenticated user information, scoped per-thread (Book 08's ThreadLocal concept) so

each concurrently-processed request has its own independent authentication state without interfering with others.

Tricky Questions

- Q: If the JWT signing key is accidentally leaked (e.g., committed to a public repository), what's the actual impact? → A: Catastrophic — anyone with the key can forge arbitrary valid tokens for any user/role, since the signature is exactly what proves a token's authenticity; this is why the signing key must be treated as a top-tier secret (Book 10, Ch.2's "never hardcode credentials," Ch.9 of this book), stored via environment variables/secrets managers, never in source code.
- Q: Does this filter's `SecurityContextHolder.getContext().setAuthentication(authToken)` persist across multiple requests from the same client? → A: No — since `SecurityContextHolder` is ThreadLocal-scoped per-request-thread (and Spring Security clears it after each request in the stateless configuration), authentication must be re-established fresh on every single request by re-validating the JWT each time; nothing is remembered between requests, which is the essence of statelessness (Book 10, Ch.7).

Coding Exercise

L1: Implement a `JwtService` generating and validating tokens, and test both valid and deliberately-tampered tokens. **L2:** Implement a `JwtAuthenticationFilter` and wire it into the security filter chain (Ch.1), testing an authenticated vs. unauthenticated request. **L3:** Reproduce the "leaked signing key lets you forge tokens" scenario conceptually — generate a token using a key you control and verify it's accepted, then explain the real-world implication in a comment. **L4 (Interview):** Explain the full JWT authentication flow from login to a subsequent authenticated request. **L5 (Senior):** Design a secrets-management strategy for the JWT signing key across dev/staging/prod (Book 12, Ch.8's profiles), ensuring it's never committed to source control. **L6 (Mastery):** Explain, from memory, why the filter always continues the chain regardless of token validity, and how that connects to the later authorization filter's role.

CHAPTER 5 — Refresh Tokens & Token Lifecycle

Telugu Explanation

Book 10, Ch.7 లో short-lived access tokens + refresh tokens pattern ప్రస్తావించాము. ఇప్పుడు దీన్ని పూర్తిగా చూద్దాం: **Access token** (short-lived, ఉదా. 15 నిమిషాలు, ప్రతి API request తో పంపబడుతుంది) + **Refresh token** (long-lived, ఉదా. 7 రోజులు, **ఒక్క** purpose కోసమే వాడతారు — కొత్త access token పొందడానికి, securely store చేయబడుతుంది).

Professional English Explanation

Book 10, Ch.7 mentioned the short-lived-access-token + refresh-token pattern. Now, in full: **Access token** (short-lived, e.g., 15 minutes, sent with every API request) + **Refresh token** (long-lived, e.g., 7 days, used for exactly **one** purpose — obtaining a new access token — and stored more securely).

Java Code

```
import org.springframework.stereotype.Service;
import java.util.*;
import java.time.Instant;

record TokenPair(String accessToken, String refreshToken) {}

@Service
class TokenLifecycleService {
    private final JwtService jwtService;
    private final Map<String, RefreshTokenRecord> refreshTokenStore = new
java.util.concurrent.ConcurrentHashMap<>();  // Book 08, Ch.4

    record RefreshTokenRecord(String username, Instant expiresAt, boolean revoked) {}

    TokenLifecycleService(JwtService jwtService) { this.jwtService = jwtService; }

    TokenPair login(String username, List<String> roles) {
        String accessToken = jwtService.generateToken(username, roles);                    //
short-lived, ~15 min

        String refreshToken = UUID.randomUUID().toString();
// opaque random string, NOT a JWT itself
        refreshTokenStore.put(refreshToken,
                new RefreshTokenRecord(username, Instant.now().plusSeconds(7 * 24 * 60 * 60),
false));   // 7 days

        return new TokenPair(accessToken, refreshToken);
    }

    TokenPair refreshAccessToken(String refreshToken) {
        RefreshTokenRecord record = refreshTokenStore.get(refreshToken);
        if (record == null || record.revoked() || record.expiresAt().isBefore(Instant.now())) {
            throw new SecurityException("Invalid or expired refresh token - must log in
again");   // 401 (Book 10, Ch.1)
        }

        // ROTATION: issue a NEW refresh token, invalidate the old one - limits the damage
window if one leaks
        refreshTokenStore.remove(refreshToken);
        String newRefreshToken = UUID.randomUUID().toString();
        refreshTokenStore.put(newRefreshToken,
                new RefreshTokenRecord(record.username(), Instant.now().plusSeconds(7 * 24 * 60
* 60), false));

        String newAccessToken = jwtService.generateToken(record.username(), List.of("USER"));
        return new TokenPair(newAccessToken, newRefreshToken);
    }

    void logout(String refreshToken) {
        refreshTokenStore.remove(refreshToken);
// immediate revocation
        // Note: any already-issued ACCESS tokens remain valid until their own short expiry -
        // this is the fundamental trade-off of stateless access tokens (Ch.4's design)
    }
}
```

Diagram — Token Lifecycle

```
LOGIN:
  Client -> Server: username/password
  Server -> Client: {accessToken (15 min), refreshToken (7 days, opaque, stored server-side)}

NORMAL API USE:
  Client -> Server: GET /api/orders, Authorization: Bearer <accessToken>
  (repeats for every request, until accessToken expires)

WHEN accessToken EXPIRES:
  Client -> Server: POST /api/auth/refresh, {refreshToken}
  Server: validates refreshToken server-side (Ch.5's store) -> issues a NEW accessToken + NEW
refreshToken (rotation)
  Server -> Client: {new accessToken, new refreshToken}
  (old refreshToken is now INVALID - rotation limits a leaked-token's usable window)

LOGOUT:
  Client -> Server: POST /api/auth/logout, {refreshToken}
  Server: removes refreshToken from the store - refresh capability revoked IMMEDIATELY
  (but any still-valid accessToken keeps working until ITS OWN short expiry - a known, accepted
trade-off)
```

Internal Working

- The **access token is a self-contained, stateless JWT** (Ch.4) — the server never needs to look it up anywhere to validate it, purely verifying its signature/expiration; the **refresh token, by contrast, is deliberately looked up server-side** (in a database or cache, here a simplified `ConcurrentHashMap`, Book 08, Ch.4) — this asymmetry is deliberate: it gives the system a genuine, immediate **revocation** capability (removing a refresh token record instantly prevents any future access-token renewal) that pure stateless JWTs alone cannot provide, while still keeping the *frequent* access-token validation path fully stateless and fast.
- **Refresh token rotation** (issuing a brand-new refresh token on every use, invalidating the old one) is a real, deliberate security hardening technique — if a refresh token is ever stolen and used by an attacker, the legitimate user's next attempt to use their (now-invalidated) original refresh token would fail, providing a detectable signal that theft occurred, and limiting a stolen token's usable window to a single use rather than its entire remaining lifetime.
- The **fundamental, accepted trade-off** noted in `logout()`'s comment — an already-issued access token remains valid until its own short expiry, even after "logout" revokes the refresh token — is exactly why access tokens are kept genuinely **short-lived** (minutes, not hours/days): it bounds the maximum damage window of this specific limitation to something operationally acceptable, a deliberate design balance between pure statelessness and the practical need for timely revocation.

Real-World Example

Telugu: Mobile banking apps ఖచ్చితంగా ఈ pattern వాడతాయి — access token కొన్ని నిమిషాలకే expire అయి, app background లో automatic గా refresh token తో renew చేస్తుంది, user కి కనిపించకుండా; "logout అన్ని devices నుండి" feature refresh token revoke చేయడం ద్వారానే పనిచేస్తుంది. English: Mobile banking apps use exactly this pattern — the access token expires within minutes, silently renewed by the

app in the background using the refresh token, invisible to the user; a "log out from all devices" feature works precisely by revoking refresh tokens server-side.

Interview Answer

"Access tokens are short-lived, stateless JWTs (Ch.4) sent with every request, requiring no server-side lookup to validate. Refresh tokens are long-lived, opaque tokens looked up server-side, used solely to obtain new access tokens — this asymmetry gives genuine revocation capability (removing a refresh token record immediately blocks renewal) while keeping frequent access-token validation stateless and fast. Refresh token rotation (issuing a new one on every use, invalidating the old) limits a stolen token's usable window. The accepted trade-off: an already-issued access token remains valid until its own short expiry even after logout, which is exactly why access tokens are kept genuinely short-lived."

Cross Questions

- Q: Why does the refresh token need server-side storage/lookup while the access token doesn't? → A: To provide genuine, immediate revocation capability — a stateless JWT alone can't be "un-issued" before its expiry, but a server-side-tracked refresh token can be removed instantly, blocking future renewals.
- Q: What problem does refresh token rotation solve? → A: It limits a stolen refresh token's usable window to a single use — if an attacker uses a stolen token, the legitimate user's subsequent attempt with the (now-invalidated) original token fails, providing a detectable signal and bounding the damage.
- Q: Why does "logout" not immediately invalidate an already-issued access token? → A: Access tokens are deliberately stateless (Ch.4) for fast, lookup-free validation — this means they can't be individually revoked before their own expiry, which is exactly why they're kept short-lived, bounding this trade-off's practical impact.

Tricky Questions

- Q: If an attacker steals BOTH a valid access token and a valid refresh token, does rotation alone stop them? → A: No — rotation limits the refresh token's *reuse* window, but the attacker can still use the stolen access token for its remaining lifetime, and could potentially use the stolen refresh token once (before the legitimate user does) to get a fresh, fully-valid token pair; genuine defense requires additional measures (device binding, anomaly detection) beyond rotation alone for a fully stolen pair.
- Q: Is storing the refresh token in a simple in-memory `ConcurrentHashMap` (as in this chapter's simplified demo) production-appropriate? → A: No — a real production system needs persistent storage (a database, Book 09/13) so refresh tokens survive application restarts, and ideally supports querying/revoking all tokens for a given user (for a "logout everywhere" feature) — the in-memory map here is a simplified teaching device, not a production pattern.

Coding Exercise

L1: Implement the login/refresh/logout flow from this chapter, using an in-memory store for simplicity. **L2:** Add refresh token rotation, and verify that using an old (already-rotated) refresh token correctly fails. **L3:** Implement a "logout from all devices" feature by tracking multiple refresh tokens per username and revoking all of them at once. **L4 (Interview):** Explain why refresh tokens are stored server-side while access tokens are not. **L5 (Senior):** Design a production-grade refresh token storage strategy using Book 13's JPA persistence, including expiry cleanup. **L6 (Mastery):** Explain, from memory, the accepted trade-off around access-token revocation at logout, and why it justifies keeping access tokens short-lived.

CHAPTER 6 — OAuth2 & Social Login Concepts

Telugu Explanation

OAuth2 అనేది **delegated authorization** framework — "Google/GitHub నా identity ని verify చేసి, ఈ app కి కొన్ని specific permissions ఇవ్వండి, నా password ఆ app కి తెలియకుండానే" అని సాధించే protocol. **Authorization Code flow** (most common, web apps కి): user Google కి redirect అవుతాడు, login చేస్తాడు, Google మన app కి ఒక **authorization code** పంపుతుంది, మన backend ఆ code ని Google కి పంపి **access token** పొందుతుంది — client-side code ఎప్పుడూ user password చూడదు.

Professional English Explanation

OAuth2 is a **delegated authorization** framework — enabling "let Google/GitHub verify my identity and grant this app specific permissions, without the app ever seeing my password." The **Authorization Code flow** (most common for web apps): the user is redirected to Google, logs in there, Google sends our app an **authorization code**, our backend exchanges that code with Google for an **access token** — our client-side code never sees the user's password at all.

Diagram — OAuth2 Authorization Code Flow

```
1. User clicks "Login with Google" on OUR app
        |
        v
2. OUR app redirects the browser to Google's login page
        |
        v
3. User logs in DIRECTLY on Google's page (never sees/enters password on OUR app)
        |
        v
4. Google redirects back to OUR app with a short-lived AUTHORIZATION CODE
        |
        v
5. OUR BACKEND (server-to-server, not the browser) exchanges that code + our
   client secret with Google for an ACCESS TOKEN (and often an ID Token, containing
   verified identity claims - name, email)
        |
        v
6. OUR app now has a token proving "Google verified this user is who they claim to be"
   and, if requested, permission to call specific Google APIs on their behalf
```

Java Code — Spring Boot OAuth2 Client Configuration

```
# application.yml (Book 12, Ch.8)
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: ${GOOGLE_CLIENT_ID}                    # Book 12, Ch.8's environment
variable pattern - NEVER hardcoded
            client-secret: ${GOOGLE_CLIENT_SECRET}
            scope: profile, email                                 # requesting ONLY these
specific permissions
```

```
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.context.annotation.*;

@Configuration
class OAuth2Config {
    @Bean
    SecurityFilterChain oauth2FilterChain(HttpSecurity http) throws Exception {
        http.oauth2Login(oauth2 -> oauth2
                .defaultSuccessUrl("/api/auth/oauth2-success", true));      // after Google
login succeeds, THIS handles it
        return http.build();
    }
}

@org.springframework.web.bind.annotation.RestController
class OAuth2SuccessController {
    @org.springframework.web.bind.annotation.GetMapping("/api/auth/oauth2-success")
    Map<String, String> handleOAuth2Success(
            @org.springframework.security.core.annotation.AuthenticationPrincipal
            org.springframework.security.oauth2.core.user.OAuth2User oauth2User) {

        String email = oauth2User.getAttribute("email");                      // Google's
verified identity claim
        String name = oauth2User.getAttribute("name");
        // Real code: find-or-create a local AppUser for this email, THEN issue OUR OWN JWT
(Ch.4) for our own API
        return Map.of("email", email, "name", name, "status", "authenticated via Google");
    }
}
```

Internal Working

- The **authorization code** exchange step (Step 5) happening **server-to-server** (our backend directly calling Google, including our confidential `client-secret`) rather than in the user's browser is a deliberate, critical security property — the browser (and any JavaScript running in it) only ever sees the short-lived authorization code, never the actual access token or our client secret; this prevents a malicious script or a network observer from intercepting the powerful, longer-lived access token during the exchange.

- Spring Security's `oauth2Login()` handles the **entire redirect dance** (Steps 1-5) automatically — your application code only needs to handle what happens **after** successful authentication (Step 6), typically finding-or-creating a corresponding local user record and, in a stateless API design, issuing your **own** JWT (Ch.4) for subsequent API calls, rather than continuing to rely on Google's token for your own app's ongoing authorization decisions.
- The `scope` configuration (`profile, email`) implements the **principle of least privilege** at the OAuth2 level — requesting only the specific permissions genuinely needed, rather than broad access; Google explicitly shows the user exactly what's being requested during the consent step, and a well-behaved application should never request more scope than it actually uses.

Real-World Example

Telugu: "Sign in with Google/GitHub/Facebook" buttons ఖచ్చితంగా ఈ OAuth2 Authorization Code flow వాడతాయి — real production apps password management burden (hashing, reset flows, breach risk) ని పూర్తిగా third-party identity provider కి delegate చేస్తాయి, user experience కూడా మెరుగుపడుతుంది.
English: "Sign in with Google/GitHub/Facebook" buttons use exactly this OAuth2 Authorization Code flow — real production apps delegate the entire password-management burden (hashing, reset flows, breach risk) to a trusted third-party identity provider, improving both security posture and user experience simultaneously.

Interview Answer

"OAuth2 is a delegated authorization framework — a user authenticates directly with a trusted provider (Google, GitHub), which then issues our application a token proving verified identity, without our app ever seeing the user's actual password. The Authorization Code flow exchanges a short-lived code for an access token server-to-server (not in the browser), keeping the powerful access token and client secret away from client-side exposure. Spring Security's `oauth2Login()` automates the entire redirect flow, letting application code focus on what happens after successful authentication — typically issuing our own JWT (Ch.4) for subsequent stateless API calls."

Cross Questions

- Q: Why does the authorization-code-to-access-token exchange happen server-to-server rather than in the browser? → A: To keep the powerful access token and the confidential client secret away from client-side/browser exposure — the browser only ever handles the short-lived, less powerful authorization code.
- Q: What does the `scope` parameter control, and why should it be minimal? → A: It specifies exactly which permissions/data the application is requesting from the provider — following the principle of least privilege, requesting only what's genuinely needed rather than broad access.
- Q: After a successful OAuth2 login via Google, does the application typically continue relying on Google's token for its own subsequent API authorization? → A: Usually no — the common pattern is to find-or-create a local user record and issue the application's own JWT (Ch.4) for its own stateless API's subsequent authorization decisions, using the OAuth2 login only for the initial identity verification step.

Tricky Questions

- Q: If a malicious actor intercepts the authorization code in the browser redirect (Step 4), can they use it to fully compromise the account? → A: Not directly by itself — exchanging the code for an access token also requires the application's confidential `client-secret` (known only server-side), so an intercepted code alone, without that secret, generally cannot complete the exchange; this is precisely why the secret staying server-side matters.
- Q: Is OAuth2 the same thing as OpenID Connect (OIDC)? → A: Not quite — OAuth2 itself is purely an *authorization* framework (delegating permission to access resources); OIDC is a thin identity layer built on top of OAuth2 specifically standardizing *authentication* (the ID Token with verified identity claims) — "social login" in practice typically uses OIDC-on-top-of-OAuth2, a nuance worth knowing precisely for senior-level discussions.

Coding Exercise

L1: Configure OAuth2 login with a test provider (or research Google's OAuth2 setup process) and trace through each step of the flow. **L2:** Implement the post-login handler that finds-or-creates a local user and issues your own JWT (Ch.4) using the OAuth2-provided email. **L3:** Research and summarize, in your own words, the distinction between OAuth2 (authorization) and OpenID Connect (authentication). **L4 (Interview):** Explain the Authorization Code flow step by step. **L5 (Senior):** Design a social-login integration strategy for an application supporting both traditional username/password and Google/GitHub login, unifying both into one internal user model. **L6 (Mastery):** Explain, from memory, why the authorization-code exchange happens server-to-server, and what security property that provides.

CHAPTER 7 — CORS Deep Dive

Telugu Explanation

CORS (Cross-Origin Resource Sharing) browsers enforce చేసే **same-origin policy** ని safely relax చేయడానికి ఒక mechanism — `frontend.example.com` నుండి JavaScript `api.example.com` కి request పంపాలంటే (వేరే origin), server explicit గా అనుమతి ఇవ్వాలి (`Access-Control-Allow-Origin` header ద్వారా). ఇది **browser-enforced** security — server-to-server calls (Postman, mobile apps) CORS కి subject కావు.

Professional English Explanation

CORS (Cross-Origin Resource Sharing) is the mechanism safely relaxing the browser-enforced **same-origin policy** — for JavaScript on `frontend.example.com` to call `api.example.com` (a different origin), the server must explicitly grant permission via the `Access-Control-Allow-Origin` header. This is **browser-enforced** security — server-to-server calls (Postman, mobile apps, `curl`) are never subject to CORS at all.

Java Code

```
import org.springframework.context.annotation.*;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import java.util.List;

@Configuration
class CorsConfig {

    @Bean
    CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(List.of("https://frontend.example.com"));       // EXPLICIT
allowlist - never "*" with credentials!
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));           // Book
10, Ch.1's HTTP methods
        config.setAllowedHeaders(List.of("Authorization", "Content-Type"));            //
headers the CLIENT is allowed to send
        config.setAllowCredentials(true);                                                //
allows cookies/auth headers cross-origin
        config.setMaxAge(3600L);                                                           //
cache the preflight result for 1 hour

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/api/**", config);
        return source;
    }
}
```

Diagram — The CORS Preflight Request

```
Browser JavaScript on https://frontend.example.com wants to:
   fetch("https://api.example.com/api/orders", { method: "POST", headers: {...} })

STEP 1 - PREFLIGHT (automatic, browser-initiated, for "non-simple" requests):
   Browser -> Server: OPTIONS /api/orders
              Origin: https://frontend.example.com
              Access-Control-Request-Method: POST
              Access-Control-Request-Headers: Authorization, Content-Type

   Server -> Browser: 200 OK
              Access-Control-Allow-Origin: https://frontend.example.com
              Access-Control-Allow-Methods: GET, POST, PUT, DELETE
              Access-Control-Allow-Headers: Authorization, Content-Type

STEP 2 - ACTUAL REQUEST (only sent if preflight was approved):
   Browser -> Server: POST /api/orders  (the REAL request, now actually sent)
   Server -> Browser: 200 OK, {...actual response...}

If the preflight response DOESN'T include the browser's origin in Access-Control-Allow-Origin,
the browser NEVER SENDS the actual request at all - it blocks it CLIENT-SIDE, in JavaScript,
before any real request reaches the server.
```

Internal Working

- CORS is **entirely a browser-side enforcement mechanism** — the actual HTTP request/response can genuinely succeed at the network/server level even when CORS blocks it; what CORS blocks is specifically **JavaScript's ability to read the response** in the browser's fetch/XHR API — this is why tools like Postman or `curl` (which aren't browsers enforcing this JavaScript-level restriction) never experience CORS errors calling the exact same API, a very common point of confusion for developers new to CORS.
- The **preflight request** (an automatic `OPTIONS` request the browser sends before certain "non-simple" requests — those using methods beyond GET/POST, or custom headers like `Authorization`) exists so the browser can check permission **before** potentially sending a request with side effects (like a `DELETE`) to a server that never intended to allow it — a real security-in-depth design, preventing a malicious cross-origin script from even attempting state-changing requests against an unwilling server.
- **`allowCredentials(true)` combined with a wildcard `allowedOrigins("*")` is explicitly disallowed** by the CORS specification itself (browsers will refuse to honor this combination) — this is a deliberate safety rail: allowing credentialed (cookie/auth-header-bearing) cross-origin requests from literally any origin would be a serious security hole, so specifying an explicit, real origin allowlist is mandatory whenever credentials are involved.

Real-World Example

Telugu: React/Angular/Vue frontend (`frontend.example.com`) Spring Boot backend API (`api.example.com`) ని call చేసేటప్పుడు, CORS configure చేయకపోతే browser console లో "CORS policy" error వస్తుంది — ఇది backend నిజంగా fail అవ్వడం కాదు, browser JavaScript response చదవడాన్ని block చేయడం. ఇది modern frontend/backend-separated architectures లో అత్యంత frequently కనిపించే, తరచుగా

misunderstood setup issue. English: A React/Angular/Vue frontend calling a separately-hosted Spring Boot API without CORS configured produces a "CORS policy" browser console error — the backend hasn't actually failed at all; the browser is blocking JavaScript from reading the response. This is, in practice, one of the most commonly encountered and most frequently misunderstood setup issues in modern frontend/backend-separated architectures.

Interview Answer

"CORS is a browser-enforced relaxation of the same-origin policy — a server must explicitly allow a given origin via `Access-Control-Allow-Origin` for browser JavaScript to read its response across origins. It's entirely browser-side: the request can succeed at the network level, but the browser blocks JavaScript from accessing the response if not permitted. Non-simple requests trigger an automatic preflight `OPTIONS` check first. Combining `allowCredentials(true)` with a wildcard origin is explicitly disallowed by the spec — an explicit origin allowlist is required whenever credentials are involved."

Cross Questions

- Q: Does a CORS error mean the server actually rejected the request? → A: Not necessarily — the server may have processed the request successfully; CORS specifically blocks the browser's JavaScript from reading the response, which is why tools like Postman/curl (not enforcing this browser rule) never see CORS errors against the same API.
- Q: What triggers a preflight OPTIONS request? → A: "Non-simple" requests — those using HTTP methods beyond GET/POST/HEAD, or including custom headers like `Authorization` — the browser checks permission via OPTIONS before sending the actual request.
- Q: Why is `allowCredentials(true)` with a wildcard origin disallowed? → A: It would allow any website anywhere to make credentialed (cookie/auth-bearing) cross-origin requests against your API — a serious security hole the CORS specification itself prevents by disallowing this specific combination.

Tricky Questions

- Q: If a malicious website tries to make a cross-origin request to a CORS-protected API without the user's browser having the right credentials/session, is CORS what stops the attack, or something else? → A: CORS specifically controls whether the malicious site's JavaScript can *read the response* — it doesn't prevent the request from being sent or processed server-side; genuine protection against unauthorized state-changing requests also relies on proper authentication/authorization (Ch.2-3) and, historically, CSRF protection (Ch.8) for cookie-based auth scenarios.
- Q: Does setting `Access-Control-Allow-Origin: *` ever fully disable CORS protection entirely? → A: For non-credentialed requests, it does allow any origin to read the response — but it still doesn't allow credentialed requests (per the disallowed-combination rule above), and it's still fundamentally a browser-side JavaScript-read restriction, not a general request-blocking mechanism.

Coding Exercise

L1: Configure CORS for a Spring Boot API allowing a specific frontend origin, and test both an allowed and a disallowed origin. **L2:** Trigger a preflight request (e.g., via a DELETE call with a custom header) and observe the OPTIONS request in browser dev tools. **L3:** Attempt to configure `allowCredentials(true)` with a wildcard origin and observe Spring Security's rejection of this configuration. **L4 (Interview):** Explain why CORS is browser-side and why Postman/curl never encounter CORS errors. **L5 (Senior):** Design the CORS configuration for a production application with a frontend on one domain and an API on a subdomain, considering credentials and multiple environments (dev/staging/prod). **L6 (Mastery):** Explain, from memory, exactly what a CORS preflight request checks and why it happens before the actual request for certain request types.

CHAPTER 8 — CSRF Deep Dive

Telugu Explanation

CSRF (Cross-Site Request Forgery) attack: user ఒక site (bank.com) కి already login అయి ఉన్నప్పుడు (cookie-based session, Book 10 Ch.3), attacker తయారు చేసిన మరో site (evil.com) hidden form/request తో bank.com కి unwanted action (money transfer వంటివి) trigger చేస్తుంది — browser **automatic గా** cookie ని ఆ request తో పంపుతుంది కాబట్టి, bank.com దాన్ని legitimate request గా భావిస్తుంది. CSRF token ఈ దాడిని prevent చేస్తుంది — attacker కి తెలియని, per-session unique token అవసరం అవుతుంది.

Professional English Explanation

CSRF (Cross-Site Request Forgery) attack: while a user is already logged into a site (bank.com, cookie-based session, Book 10 Ch.3), an attacker's separate site (evil.com) triggers an unwanted action (like a money transfer) via a hidden form/request against bank.com — the browser **automatically** attaches the session cookie to that request, so bank.com treats it as legitimate. A CSRF token prevents this — requiring a per-session, unique value the attacker's site cannot know or supply.

Diagram — The CSRF Attack (and the Fix)

```
WITHOUT CSRF PROTECTION:
  1. User logs into bank.com - browser stores a session cookie
  2. User visits evil.com (in the SAME browser, same session still active)
  3. evil.com's page contains: <form action="https://bank.com/transfer" method="POST">
                                 <input name="amount" value="10000">
                                 <input name="to" value="attacker-account">
                               </form>
     (auto-submitted via JavaScript, or a tricked click)
  4. Browser sends the POST to bank.com, AUTOMATICALLY attaching the session cookie
     (the browser doesn't know/care that the FORM came from evil.com!)
  5. bank.com sees a valid session cookie -> processes the transfer as if the USER intended it

WITH CSRF PROTECTION:
  1. bank.com's legitimate page embeds a CSRF token (unique, tied to the user's session)
       <input type="hidden" name="_csrf" value="a1b2c3...">
  2. evil.com has NO WAY to know/obtain this token (same-origin policy prevents reading
bank.com's page)
  3. evil.com's forged request is missing the correct _csrf token
  4. bank.com REJECTS the request - 403 Forbidden
```

Java Code

```
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.csrf.CookieCsrfTokenRepository;
import org.springframework.context.annotation.*;

@Configuration
class CsrfAwareSecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            // For a TRADITIONAL server-rendered app using session/cookie auth: CSRF protection
MATTERS, keep it ENABLED
            .csrf(csrf ->
csrf.csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse()));
        return http.build();
    }
}

@Configuration
class StatelessApiSecurityConfig {

    @Bean
    SecurityFilterChain statelessFilterChain(HttpSecurity http) throws Exception {
        http
            // For a STATELESS JWT API (Ch.4): CSRF protection is UNNECESSARY and safely
disabled
            .csrf(csrf -> csrf.disable());
        return http.build();
    }
}
```

Internal Working

- CSRF fundamentally exploits **automatic credential attachment** — specifically, that browsers automatically attach cookies to *any* request to a given domain, regardless of which site's page initiated that request; this is precisely why CSRF is a real, serious risk for **cookie/session-based** authentication (Book 10, Ch.3) but is **structurally not a risk** for JWT-based Bearer-token authentication (Ch.4) — a JWT must be **explicitly** attached by client-side JavaScript to the `Authorization` header, and a malicious cross-origin page has no automatic mechanism to make the victim's browser do that (and same-origin policy/CORS, Ch.7, prevents it from reading the legitimate token to steal and reuse it either).
- The CSRF token defense works because it relies on something the browser does **not** automatically attach: the attacker's forged form can trigger the browser to *send* the request (with cookies automatically included), but has no way to know or include the correct, session-specific CSRF token value, since same-origin policy prevents `evil.com`'s JavaScript from reading `bank.com`'s page content to extract it.
- **Disabling CSRF for a stateless JWT API (Ch.1's demo, this chapter's `StatelessApiSecurityConfig`) is the CORRECT choice, not a security shortcut** — since there's no automatically-attached credential for CSRF to exploit in the first place, the protection would

be pure, purposeless overhead; this directly resolves Ch.1's tricky question about whether disabling CSRF there was a regression.

Real-World Example

Telugu: Traditional server-rendered web applications (session cookie-based login, Thymeleaf/JSP వంటివి) కి CSRF protection **తప్పనిసరి** — cookie automatic గా browser పంపుతుంది కాబట్టి. Modern SPA (React/Angular) + stateless JWT REST API architectures కి CSRF protection అవసరం లేదు, ఎందుకంటే token cookie కాదు, JavaScript explicit గా attach చేయాలి. English: Traditional server-rendered web applications (session-cookie-based login, Thymeleaf/JSP) absolutely require CSRF protection, since the cookie is automatically attached by the browser. Modern SPA (React/Angular) + stateless JWT REST API architectures don't need it, since the token isn't a cookie — it must be explicitly attached by JavaScript, structurally immune to the classic CSRF attack pattern.

Interview Answer

"CSRF exploits the browser's automatic attachment of cookies to any request against a domain, regardless of which site initiated it — letting an attacker's page trigger unwanted authenticated actions against a site the victim is logged into. CSRF tokens defend against this by requiring a session-specific value the attacker's page has no way to read or supply, thanks to same-origin policy. This attack is real and serious for cookie/session-based authentication, but structurally doesn't apply to JWT Bearer-token authentication — since a JWT must be explicitly attached to a header by client-side JavaScript, never automatically like a cookie — which is exactly why disabling CSRF protection for a stateless JWT API is correct, not a security shortcut."

Cross Questions

- Q: What specific browser behavior does CSRF exploit? → A: Automatic attachment of cookies to any request to a given domain, regardless of which site's page actually initiated that request.
- Q: Why does a CSRF token actually stop the attack? → A: The attacker's page cannot read or know the legitimate site's session-specific CSRF token value, due to same-origin policy — so its forged request is missing the correct token and gets rejected.
- Q: Why is CSRF protection unnecessary for a JWT Bearer-token API? → A: A JWT isn't automatically attached by the browser like a cookie — it must be explicitly added to a request header by client-side JavaScript, which a malicious cross-origin page has no automatic mechanism to trigger or read.

Tricky Questions

- Q: If a stateless JWT API stores the JWT in a cookie (instead of sending it via the Authorization header, perhaps for convenience), does it become vulnerable to CSRF again? → A: Yes — this is a genuine, real trap: if the token is stored in a cookie and the browser automatically attaches it, the same CSRF attack pattern applies again, even though it's technically a JWT; CSRF risk is fundamentally about *automatic credential attachment*, not about the specific token format used.
- Q: Does CORS (Ch.7) provide any CSRF protection on its own? → A: Not really — CORS controls whether cross-origin JavaScript can *read* a response, but a CSRF attack often doesn't need to

read the response at all (a state-changing side effect, like a money transfer, succeeds regardless of whether the attacker's page can see the result) — CORS and CSRF protect against related but distinct threat models.

Coding Exercise

L1: Reproduce (in a safe, local test environment) a simplified CSRF scenario using a session-cookie-authenticated endpoint without CSRF protection. **L2:** Add CSRF token protection and verify the same forged-request pattern is now rejected. **L3:** Explain, in a comment, why storing a JWT in a cookie (rather than sending it via header) would reintroduce CSRF risk. **L4 (Interview):** Explain the CSRF attack mechanism and why CSRF tokens defend against it. **L5 (Senior):** Review a hybrid application using both session-cookie auth for its web UI and JWT Bearer tokens for its mobile API — design the correct CSRF configuration for each part. **L6 (Mastery):** Explain, from memory, why disabling CSRF protection is correct for a properly-implemented stateless JWT API, but would be a serious mistake if that same JWT were instead stored in a cookie.

CHAPTER 9 — Production Security Practices

Telugu Explanation

ఈ chapter లో production Spring Security deployments లో ముఖ్యమైన additional practices చూద్దాం: **HTTPS everywhere** (TLS లేకుండా JWT/passwords plaintext గా network mీద కనిపిస్తాయి), **Security headers** (`HSTS`, `X-Content-Type-Options`, `X-Frame-Options` — Spring Security default గా చాలా వరకు add చేస్తుంది), **Secrets management** (Book 12, Ch.8's environment variables, dedicated secrets managers), **Rate limiting** (login endpoints కి, brute-force attacks నివారించడానికి — Book 16 లో పూర్తిగా).

Professional English Explanation

This chapter covers additional important practices for production Spring Security deployments: **HTTPS everywhere** (without TLS, JWTs/passwords are visible in plaintext on the network), **Security headers** (`HSTS`, `X-Content-Type-Options`, `X-Frame-Options` — Spring Security adds most of these by default), **Secrets management** (Book 12, Ch.8's environment variables, dedicated secrets managers), **Rate limiting** (on login endpoints, to prevent brute-force attacks — covered fully in Book 16).

Java Code — Security Headers & Production Checklist

```
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.context.annotation.*;

@Configuration
class ProductionSecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .headers(headers -> headers
                .httpStrictTransportSecurity(hsts -> hsts.maxAgeInSeconds(31536000))    //
force HTTPS for 1 year, even on downgrade attempts
                .contentTypeOptions(withDefaults -> {})                                    //
prevents MIME-sniffing attacks
                .frameOptions(frame -> frame.deny())
// prevents clickjacking (embedding in an <iframe>)
            )
            .requiresChannel(channel -> channel.anyRequest().requiresSecure());
// reject plain HTTP entirely
        return http.build();
    }
}
```

Production Security Checklist

```
[ ] HTTPS enforced everywhere - no plaintext HTTP for anything carrying credentials/tokens
[ ] JWT signing key stored in a secrets manager / environment variable, NEVER in source code
(Book 12, Ch.8)
[ ] Passwords hashed with BCrypt (or Argon2), never plaintext, never a fast general-purpose
hash (Ch.2)
[ ] Access tokens short-lived (minutes); refresh tokens rotated and revocable (Ch.5)
[ ] CORS configured with an explicit origin allowlist, never wildcard + credentials (Ch.7)
[ ] CSRF protection correctly enabled for cookie-based auth, correctly disabled for pure
Bearer-token APIs (Ch.8)
[ ] Rate limiting on login/registration endpoints - prevents brute-force credential attacks
(Book 16 full detail)
[ ] Generic error messages for failed login ("invalid credentials", NOT "user not found" vs
"wrong password" -
    the latter leaks which usernames exist, a real information-disclosure risk)
[ ] Sensitive data (passwords, tokens, PII) NEVER logged (Book 12, Ch.9's logging discipline)
[ ] Dependency versions kept current - known CVEs in security libraries are a real, actively-
exploited risk
[ ] Security-relevant configuration reviewed in code review, not just functional correctness
```

Internal Working

- **HSTS (`HTTP Strict Transport Security`)** solves a subtle real attack: even if your server correctly redirects HTTP → HTTPS, the **very first** request a browser makes to a domain could still go out over plain HTTP before that redirect happens, giving an attacker on the network a brief window to intercept it (a "downgrade attack") — HSTS tells the browser, for a specified duration, to **never even attempt** a plain HTTP connection to this domain again, closing that window entirely after the first successful HTTPS visit.
- The **generic login error message** practice directly connects to Book 04's exception-handling philosophy (never leak internal details) applied specifically to authentication: returning "user not found" for a nonexistent username but "wrong password" for an existing one lets an attacker efficiently enumerate which usernames are valid in your system (a real information-disclosure vulnerability, distinct from but related to the credentials themselves), which is why production systems return one identical, generic message regardless of which specific thing was wrong.
- **Rate limiting login endpoints** is what actually makes offline brute-force resistance (Ch.2's slow BCrypt hashing) meaningfully complete — BCrypt slows down an attacker who has *stolen the password hash database*; rate limiting slows down an attacker who's instead directly guessing passwords *through your live login endpoint*, a different attack vector requiring a different, complementary defense (full rate-limiting implementation detail in Book 16).

Real-World Example

Telugu: Real production security incidents చాలావరకు "ఒక్క weak link" వల్ల జరుగుతాయి — password hashing సరిగ్గా చేసినా, JWT secret hardcoded గా repository లో committed అయితే, మొత్తం security పనికిరాకుండా పోతుంది. ఈ chapter యొక్క checklist అంతా ఒక్కదాన్ని కాకుండా, **అన్నింటినీ** కలిపి సరిగ్గా చేయడమే production-grade security. English: Real production security incidents most often stem from just one weak link — properly hashing passwords means little if the JWT signing key is hardcoded and committed to the repository, undermining the entire security posture regardless of what else was

done correctly. This chapter's checklist emphasizes that production-grade security requires getting **all** of these practices right together, not excelling at just one while neglecting the others.

Interview Answer

"Production Spring Security requires HTTPS everywhere (HSTS to close the downgrade-attack window on the first request), proper secrets management for signing keys, BCrypt password hashing, short-lived rotatable tokens, explicit CORS origin allowlists, correctly-scoped CSRF protection, and rate limiting on login endpoints to complement offline brute-force resistance. A critical, often-overlooked practice: generic login error messages, since distinguishing 'user not found' from 'wrong password' leaks which usernames exist in the system — an information-disclosure risk directly connecting to Book 04's 'never leak internal details' exception-handling philosophy."

Cross Questions

- Q: What specific attack does HSTS prevent that a simple HTTP-to-HTTPS redirect doesn't fully close? → A: A downgrade attack during the very first request to a domain — before any redirect has happened, that first request could still be intercepted over plain HTTP; HSTS tells the browser to never attempt plain HTTP again after the first successful HTTPS visit.
- Q: Why is returning "user not found" vs "wrong password" as distinct login error messages a security risk? → A: It lets an attacker efficiently determine which usernames are valid in the system (username enumeration), a real information-disclosure vulnerability — a single generic message avoids this.
- Q: Why is rate limiting needed on login endpoints even with properly-hashed (BCrypt) passwords? → A: BCrypt's slowness defends against an attacker who has stolen the password hash database and is brute-forcing offline; rate limiting defends against a different attack vector — an attacker directly guessing passwords through the live login endpoint itself.

Tricky Questions

- Q: If a company properly implements every practice in this chapter's checklist except keeping dependency versions current, are they still meaningfully secure? → A: Not necessarily — a known, unpatched CVE in a security-relevant library (Spring Security itself, a JWT library) can be independently exploitable regardless of how well everything else is configured; dependency currency is a genuinely necessary, not merely nice-to-have, part of the full security posture.
- Q: Does enabling HTTPS alone (without HSTS) provide meaningfully weaker protection than HTTPS + HSTS? → A: Yes, specifically for that narrow "very first connection" downgrade-attack window — HTTPS alone protects all subsequent, correctly-redirected traffic, but HSTS closes the specific gap where an attacker positioned on the network could intercept that very first plain-HTTP request before any redirect occurs.

Coding Exercise

L1: Configure HSTS, frame options, and content-type-options headers on a Spring Security filter chain, and verify the response headers. **L2:** Implement a generic login error response that doesn't

distinguish "user not found" from "wrong password." **L3:** Walk through the full production security checklist for a hypothetical application, marking which items you'd need to implement and researching any you're unfamiliar with. **L4 (Interview):** Explain what HSTS protects against that a plain HTTP-to-HTTPS redirect doesn't. **L5 (Senior):** Conduct a security review of a hypothetical login flow, identifying at least 3 checklist violations and their fixes. **L6 (Mastery):** Explain, from memory, why BCrypt hashing and rate limiting are complementary, not redundant, defenses against password-guessing attacks.

CHAPTER 10 — Mini Project: JWT-Secured REST API

Goal

Combine every concept from this book into one complete, production-shaped, JWT-secured Spring Boot REST API (building directly on Book 12/13's REST API and JPA work).

Requirements

1. **Full security filter chain configuration** (Ch.1): stateless session policy, correctly-ordered `authorizeHttpRequests` rules for public, authenticated, and role-restricted endpoints.
2. **`UserDetailsService` + BCrypt** (Ch.2), backed by a real Book 13 JPA `UserRepository`, with a registration endpoint that never logs raw passwords.
3. **Role-based AND ownership-based authorization** (Ch.3): at least one `@PreAuthorize("hasRole(...)")` and one ownership-based `@PreAuthorize` check, correctly avoiding the self-invocation pitfall.
4. **Full JWT authentication flow** (Ch.4): login endpoint issuing a token, a custom filter validating it on every request.
5. **Refresh token lifecycle** (Ch.5) with rotation, backed by Book 13's JPA persistence (not the simplified in-memory map from this chapter's demo).
6. **Correct CORS configuration** (Ch.7) for a specified frontend origin, with credentials support.
7. **Correct CSRF decision** (Ch.8), justified in a comment given the stateless JWT design.
8. **Production security headers** (Ch.9): HSTS, frame options, content-type options, plus a generic login error message.
9. **Testing** (Book 12, Ch.11 preview / full detail in Book 15): at least one test verifying an unauthenticated request is rejected (401) and one verifying an authenticated-but-unauthorized request is rejected (403).

Concepts Reinforced

Every chapter in this book — the filter chain architecture, authentication and password hashing, role/ownership-based authorization, JWT and refresh tokens, OAuth2 concepts, CORS, CSRF, and production security practices — applied together in one complete, realistic secured REST API, building directly on Books 09-13's foundations.

Stretch Goal

Add Google OAuth2 login (Ch.6) as an alternative authentication method alongside username/password, unifying both into the same local user model and JWT-issuance flow.

📌 FINAL REVISION NOTES

- **Filter chain**: Spring Security = one servlet Filter (Book 10, Ch.4) containing an ordered internal chain; `authorizeHttpRequests` rules evaluated top-to-bottom, first-match-wins — specific rules before broad ones.
- **Authentication**: `UserDetailsService` connects to your data; BCrypt = deliberately slow + auto-salted, one-way hash; never store or log plaintext passwords.
- **Authorization**: URL-level + method-level (`@PreAuthorize` / `@PostAuthorize` , AOP-based, Book 11 Ch.8) = defense in depth; same self-invocation limitation applies; `@PostAuthorize` for decisions needing the return value (e.g., ownership).
- **JWT auth**: signature verification IS the security; filter always continues the chain; `SecurityContextHolder` is ThreadLocal-scoped per request.
- **Refresh tokens**: access token = stateless JWT, short-lived; refresh token = server-side-tracked, revocable, rotated on use; logout revokes refresh capability, not already-issued access tokens (accepted trade-off).
- **OAuth2**: delegated authorization; authorization-code exchange happens server-to-server, keeping the access token/secret off the browser; typically followed by issuing your own JWT.
- **CORS**: browser-side only (blocks JS reading the response, not the request itself); preflight for non-simple requests; `allowCredentials(true)` + wildcard origin is disallowed by spec.
- **CSRF**: exploits automatic cookie attachment; real risk for session/cookie auth, structurally absent for header-based JWT auth (but returns if the JWT is stored in a cookie instead).
- **Production practices**: HTTPS + HSTS, secrets management, short-lived rotatable tokens, explicit CORS allowlist, generic login errors (no username enumeration), rate limiting (complements BCrypt, doesn't replace it), current dependencies.

🗒️ CHEAT SHEET

```
Filter chain: 1 servlet Filter (Book 10 Ch.4) internally ordered | authorizeHttpRequests: top-
to-bottom, FIRST MATCH WINS
UserDetailsService: your data -> Spring Security | BCrypt: slow(brute-force resist) +
salted(rainbow-table resist) + one-way
Authorization: URL-level + method-level(@PreAuthorize/@PostAuthorize, AOP, Book 11 Ch.8) =
defense in depth
  SAME self-invocation limitation (this.method() bypasses check!) | @PostAuthorize = needs
return value (ownership check)
JWT: signature = tamper-evidence, NOT secrecy | filter ALWAYS continues chain |
SecurityContextHolder = ThreadLocal/request
Refresh tokens: access=stateless JWT(short) | refresh=server-tracked,revocable,ROTATED on use
  logout revokes refresh, NOT already-issued access tokens (bounded by short expiry - accepted
trade-off)
OAuth2: delegated auth | code-exchange = SERVER-TO-SERVER (secret+token never hit browser) |
issue OWN JWT after
CORS: browser-side ONLY (blocks JS reading response) | preflight=OPTIONS for non-simple
requests
  allowCredentials(true)+wildcard origin = DISALLOWED by spec
CSRF: exploits AUTOMATIC cookie attachment | real risk=cookie/session auth | NOT a risk=header-
based JWT
  (UNLESS JWT stored in a cookie - then risk returns!)
Production checklist: HTTPS+HSTS | secrets mgmt | BCrypt | short+rotated tokens | explicit CORS
allowlist
  correct CSRF scoping | rate limiting(complements BCrypt) | generic login errors(no username
enumeration)
  never log secrets | current dependencies
```

🎤 INTERVIEW QUESTION BANK — Spring Security

Beginner

1. What is the Spring Security filter chain?
2. Why should passwords never be stored in plaintext?
3. What is the difference between authentication and authorization in Spring Security terms?
4. What is a JWT, and how is it validated?
5. What is CORS, and why does it exist?

Intermediate 6. Explain why authorizeHttpRequests rule order matters, with an example of a misconfiguration. 7. Explain why BCrypt is deliberately slow and automatically salted. 8. Explain the difference between @PreAuthorize and @PostAuthorize. 9. Explain the access-token/refresh-token pattern and why refresh tokens are stored server-side. 10. Explain the CSRF attack and why it doesn't apply to JWT Bearer-token APIs.

Advanced 11. Explain the AOP self-invocation limitation as it applies to @PreAuthorize, and its security implication. 12. Explain refresh token rotation and what attack scenario it mitigates. 13. Explain the OAuth2 Authorization Code flow and why the code exchange happens server-to-server. 14. Explain why disabling CSRF protection is correct for a stateless JWT API but would be a mistake if the JWT were stored in a cookie. 15. Explain the username-enumeration risk in login error messages and how to avoid it.

Senior/Architect 16. Design the complete security architecture (authentication, authorization, token lifecycle) for a new multi-tenant SaaS REST API. 17. Review a security configuration with misordered authorization rules exposing an admin endpoint — diagnose and fix it. 18. Design a secrets-management and key-rotation strategy for JWT signing keys across a multi-environment deployment. 19. Explain end-to-end what happens from a login request to a subsequent authorized API call, covering every filter and mechanism involved. 20. Design a defense-in-depth authorization strategy combining URL-level and method-level security for a system with both role-based and ownership-based access rules.

(Full short/professional/deep-senior answer scaffolding for every question across the series lives in Book 23 — Java Interview Master Book.)

🔁 CROSS-QUESTION ENGINE — Sample Chains

Q: How does JWT authentication work in Spring Security? → Q: What does the filter do when the token is invalid? → Q: How is SecurityContextHolder scoped? → Q: Why is a refresh token needed at all? → Q: Why is the refresh token stored differently from the access token? → Q: What does rotation protect against?

Q: Why does Spring Security use BCrypt for passwords? → Q: What makes BCrypt different from a fast hash like SHA-256 for this purpose? → Q: What does the embedded salt protect against? → Q: Can the original password ever be recovered from the hash? → Q: What complementary defense is still needed against live login brute-forcing?

Q: What is CSRF, and when does it matter? → Q: What specific browser behavior does it exploit? → Q: Why doesn't it apply to a stateless JWT API using the Authorization header? → Q: What would make it apply again, even with a JWT? → Q: How does a CSRF token actually stop the attack?

🏋️ CONSOLIDATED EXERCISES (All Levels)

L1 — Beginner: Redo each chapter's L1 exercise unaided. **L2 — Intermediate:** Redo each chapter's L2/L3 exercises, explaining every security mechanic out loud in Telugu. **L3 — Advanced:** Build a complete authentication + role-based authorization flow from scratch, including registration, login, and a protected endpoint. **L4 — Interview:** Answer all 20 Interview Bank questions from memory, under 90 seconds each. **L5 — Senior:** Complete the Chapter 10 mini project fully, including the OAuth2 stretch goal. **L6 — Mastery:** Teach Chapters 4 (JWT auth), 5 (refresh tokens), and 8 (CSRF) out loud, from memory, using fresh examples.

🗓️ ONE-DAY REVISION PLAN (≈5.5 hours)

Time	Focus
0:00–0:30	Ch.1: Filter chain architecture — memorize the rule-ordering risk
0:30–1:00	Ch.2: Authentication/password hashing
1:00–1:30	Ch.3: Authorization/method security — the self-invocation gotcha
1:30–1:45	Break
1:45–2:30	Ch.4: JWT authentication — the highest-density interview block
2:30–3:00	Ch.5: Refresh tokens
3:00–3:30	Ch.6: OAuth2 concepts
3:30–4:00	Ch.7: CORS deep dive
4:00–4:30	Ch.8: CSRF deep dive — re-read the "JWT in a cookie" trap twice
4:30–5:00	Ch.9: Production practices — recreate the checklist from memory
5:00–5:30	Interview Bank — answer all questions from memory

🗓️ ONE-WEEK MASTER REVISION PLAN

Day	Focus
1	Ch.1–2 (filter chain, authentication) — set up a basic secured Spring Boot app
2	Ch.3 (authorization) — build role-based and ownership-based method security
3	Ch.4–5 (JWT, refresh tokens) — build the full stateless authentication flow
4	Ch.6 (OAuth2) — configure and trace a social login flow
5	Ch.7–8 (CORS, CSRF) — reproduce both mechanisms and their fixes
6	Ch.9 + Mini Project — build the full JWT-secured REST API
7	Full mock interview: all 20 bank questions + all cross-question chains, in Telugu and English, without notes

✅ FINAL MASTERY CHECKLIST

- ☐ I can explain the Spring Security filter chain and the rule-ordering risk.
- ☐ I can implement UserDetailsService and BCrypt password hashing correctly.
- ☐ I can apply role-based and ownership-based method security, avoiding the self-invocation pitfall.
- ☐ I can implement full JWT-based stateless authentication.
- ☐ I can implement refresh token rotation and explain the logout trade-off.
- ☐ I can explain the OAuth2 Authorization Code flow.
- ☐ I can correctly configure CORS, including the credentials/wildcard restriction.
- ☐ I can explain CSRF and correctly decide when to enable/disable it.
- ☐ I can apply the production security checklist to a real application.
- ☐ I built the JWT-Secured REST API mini project, including the OAuth2 stretch goal.
- ☐ I answered the full Interview Question Bank confidently in both Telugu and English.

Once every box is checked, you are ready for `15_Java_Testing.md`.